

ATLAS

THE SOVEREIGN TRAVEL & WHOLESALE BANKING CLUB

Junior Software Engineer & Technical Operations

Specialized Operations Playbook • Software Architecture & Next.js 14

EXECUTIVE SPECIFICATIONS & COMPLIANCE CLEARANCE

- **Classification:** Strictly Confidential (Closed-Loop Enterprise Operations)
- **Version:** 3.4.0 Production Build (2026 Edition)
- **Standard:** Rate Parity Non-Disclosure & B2B Wholesale Distribution Licensure
- **Audited:** Verified Against Live Bedbank & FinTech Gateway Endpoints
- **Distribution:** Operations Leads, Platform Engineers & Board of Directors

© 2026 ATLAS Travel & Sovereign Banking Network. All Rights Reserved.

Junior Software Engineer & Technical Operations

Target Role: Software engineers, frontend/full-stack developers, and technical support staff responsible for maintaining and extending the ATLAS codebase.

Executive Summary: The comprehensive engineering guide to the Next.js 14 App Router architecture, TypeScript models, state management, hotel supplier adapters, PDF generator engines, and CI/CD pipelines.

CHAPTER 1 Architecture Overview & Next.js 14 App Router

🕒 15 MIN READ

1.1 Tech Stack Summary

- **Framework:** Next.js 14.2+ (App Router architecture)
- **Language:** TypeScript (Strict mode enabled)
- **Styling:** Tailwind CSS with custom luxury color palette (`amber-400` , `slate-900` , `emerald-400`)
- **Icons:** Lucide React
- **AI Integration:** Google Gemini API via `@google/genai`
- **State Management:** React Contexts (`AuthContext.tsx` , `CurrencyContext.tsx`)

1.2 Directory Structure

```
src/
├─ app/
│  ├─ layout.tsx      # Root layout with Currency & Auth Providers
│  ├─ page.tsx        # High-converting luxury public landing page
│  ├─ admin/page.tsx  # Comprehensive multi-tab operations console
│  └─ members/page.tsx # Gated members-only luxury wholesale hub
├─ api/
│  ├─ concierge/      # AI Concierge & VIP Webhook routes
│  ├─ bookings/       # Voucher generator & checkout APIs
│  ├─ cards/          # Apple/Google Wallet pass generation
│  └─ admin/          # Admin education & academy tutors
├─ components/        # Reusable UI components (Navbar, Modals, Cards)
└─ lib/               # Business logic, mock datasets, and adapters
```

MANDATORY OPERATOR CHECKLIST

4 ACTION ITEMS

- ✓ Clone repository and install dependencies with `npm install`
- ✓ Run development server with `npm run dev` on port 3000/3005
- ✓ Inspect `CurrencyContext` and `AuthContext` state lifecycles
- ✓ Review TypeScript interfaces in `src/lib/hotelData.ts`

2.1 The Unified Hotel Adapter Pattern

All hotel suppliers (Hotelbeds, WebBeds, Direct API, Scraped wholesale) must implement the unified `HotelSupplierAdapter` interface to ensure zero-breaking-change integration.

```
export interface HotelSupplierAdapter {
  supplierId: string;
  name: string;
  searchHotels(query: HotelSearchQuery): Promise<HotelOffer[]>;
  getHotelDetails(hotelId: string): Promise<HotelDetails>;
  verifyRateParity(hotelId: string, wholesaleRate: number): Promise<RateParityCheck>;
  createBooking(bookingRequest: BookingPayload): Promise<BookingConfirmation>;
}
```

2.2 Adding a New Supplier Step-by-Step

- Create a new file in `src/lib/adapters/.ts`.
- Implement rate normalization (convert supplier raw currencies to base USD).
- Apply the ATLAS wholesale spread calculation:

`Savings = (Public OTAs Average - Wholesale Net Rate)`

- Register the adapter in `src/lib/hotelData.ts`.

MANDATORY OPERATOR CHECKLIST

4 ACTION ITEMS

- ✓ Implement `HotelSupplierAdapter` interface for new supplier
- ✓ Add error handling and timeout fallbacks (max 3500ms)
- ✓ Write unit test for rate normalization across multi-currencies
- ✓ Register adapter in hotelData registry

3.1 B2B Wholesale Voucher Anatomy

When a member completes a reservation:

- The server generates a unique B2B Wholesale Voucher at `/api/bookings/voucher`.
- The voucher embeds:
 - Official B2B Bedbank Confirmation Reference.
 - Hotel address, check-in instructions, and meal plans.
 - Dynamic QR code for instant front-desk verification.
 - **Rate Parity Non-Disclosure Notice**: Legally protects the wholesale net rate from public disclosure.

3.2 QR Code & PDF Rendering Pipeline

The API route constructs clean, print-ready HTML with embedded CSS and renders dynamic SVG QR codes directly without external binary dependencies.

MANDATORY OPERATOR CHECKLIST

3 ACTION ITEMS

- ✓ Test voucher generation at `/api/bookings/voucher?bookingId=ATL-9842`
- ✓ Verify QR code scans accurately to the member verification URL
- ✓ Confirm Rate Parity legal disclaimer renders on all voucher outputs

4.1 Resiliency Best Practices

- **Idempotency Keys**: All financial transactions (card reloads, dividend payouts) must pass a UUID idempotency key to prevent double charging.
- **Graceful Fallbacks**: If a bedbank API times out (>4s), the search engine gracefully falls back to cached inventory or secondary suppliers without crashing the UI.
- **Structured Error Responses**: All API routes must return consistent JSON:

```
{ "error": true, "message": "User friendly message", "code": "SUPPLIER_TIMEOUT" }
```

MANDATORY OPERATOR CHECKLIST

3 ACTION ITEMS

- ✓ Audit all API routes for structured try/catch blocks
- ✓ Verify idempotency key validation on payment endpoints
- ✓ Monitor Next.js server logs for uncaught promise rejections

5.1 Deployment Pipeline

- **Production Host:** Vercel / Google Cloud Run
- **Source Control:** GitHub (`master` branch auto-deploys to production)
- **Pre-Flight Check:** Always execute `npm run build` locally before pushing to ensure zero TypeScript errors or missing imports.

5.2 Environment Variables Configuration

Ensure all production environment variables are configured in the deployment dashboard (`GEMINI_API_KEY` , `STRIPE_SECRET_KEY` , `TELEGRAM_BOT_TOKEN` , etc.).

MANDATORY OPERATOR CHECKLIST

3 ACTION ITEMS

- ✓ Run `npm run build` and confirm clean 0-error output
- ✓ Verify environment variables in production settings
- ✓ Push code to master and confirm deployment success